

Contents

Guide 2 — The Recommendation Engine (TOPSIS), simple and how to defend it

1. The idea, in three sentences

2. The method, and why (everything is published)

3. How it works — the steps

4. The formula, on a real example

5. The files (one line each)

6. Jury questions — ready answers

1

1

1

2

2

2

Guide 2 — The Recommendation Engine (TOPSIS), simple and how to defend it

A short, plain guide to how Mobile Match Algeria recommends plans: the idea, the method (fully researched), how it works, the formula on a real example, the files, and ready answers for the jury.

1. The idea, in three sentences

The user tells us their **budget**, how much **data** they need, optional filters (operator / type / network), and **ranks** what matters most (price, data). The engine throws away every plan that fails the hard limits, then ranks the rest by **how close each plan is to the “ideal” plan**, using a standard published method (TOPSIS). It uses no AI and no other users’ data — only the profile and the plan attributes, so it works from day one.

2. The method, and why (everything is published)

- **TOPSIS** — *Technique for Order of Preference by Similarity to Ideal Solution* (Hwang & Yoon, 1981; recent guide: Taherdoost & Madanchian, 2023). It ranks options by their **Euclidean distance** to an **ideal** plan (the best value on every criterion) and an **anti-ideal** plan (the worst on every criterion).
- **ROC weights** — *Rank-Order Centroid* (Barron & Barrett, 1996). Turns the user’s **ranking** of the criteria into numeric weights (price #1, data #2 → **0.75, 0.25**), so no weight is hand-picked.

Why this method: it’s **content-based**, so it needs no usage history and works at launch (cold-start); the whole thing is a **published formula** (nothing invented); and it answers the classic supervisor question — “*what are you calculating, the Euclidean distance?*” — with “**yes: the distance to the ideal plan.**”

In one line: *filter out what doesn’t fit, then rank by closeness to the ideal plan (TOPSIS), with the weights coming from your ranking (ROC).*

3. How it works — the steps

1. **Hard filters** — drop any plan over budget or with the wrong operator / type / network. (Budget is a strict ceiling.)
2. **Weights** — turn your criterion ranking into weights with ROC.
3. **Table** → **normalise** → **weight** — put the surviving plans in a table (rows = plans, columns = criteria), scale each column, multiply by the weights.
4. **Ideal & anti-ideal** — the best value in each column (cheapest price, most data) and the worst.

5. **Distances** → **closeness** — each plan's Euclidean distance to the ideal (S_+) and to the anti-ideal (S_-); closeness $c = S_+ / (S_+ + S_-)$, a number in 0–1.
 6. **Rank by C**, return the top 3; $c \times 100$ is the match %.
-

4. The formula, on a real example

weights (ROC): rank the criteria → $w_k = (1/n) * \sum_{i=k..n} (1/i)$
 price #1, data #2 → $w = 0.75, 0.25$

TOPSIS per plan:

```
normalise: r_ij = x_ij / sqrt( sum_i x_ij^2 )
weight:    v_ij = w_j * r_ij
ideal A+ (best per column), anti-ideal A- (worst per column)
distances: S+ = sqrt( sum_j (v_ij - A+_j)^2 )    S- = sqrt( sum_j (v_ij - A-_j)^2 )
closeness: C = S- / (S+ + S-)    (0 = worst, 1 = best)
```

rank by C, highest first

Worked example. Budget 2000 DA, need 10 GB, you rank **price #1, data #2** (so weights 0.75 / 0.25). Three in-budget plans:

Plan	Price	Data
A	1000 DA	8 GB
B	1100 DA	15 GB
C	600 DA	4 GB

TOPSIS measures each plan's distance to the ideal (cheapest **and** most data) and to the anti-ideal, then computes closeness:

- **C → 0.60, B → 0.40, A → 0.26**, so the ranking is **C, then B, then A**.
- **C wins**: under the 0.75 price weight its low price puts it closest to the ideal.
- **B beats A**: they are close on price, so the higher data of B (15 GB) pulls it nearer the ideal.

If you instead ranked **data #1**, the data weight would rise to 0.75 and the high-data plan **B** would move up — that is how the ROC weights steer the trade-off.

5. The files (one line each)

- [src/lib/recommendation.ts](#) — the engine: hard filters, rocWeights(), topsis(), and the top-N. The whole formula above lives here.
- [src/app/api/recommendations/route.ts](#) — the API the page calls; validates the profile, rate-limited (60 req/min).
- [src/app/recommend/page.tsx](#) — the screen: the profile form and the ranked cards (each with its savings).
- [src/app/profile/page.tsx](#) — where a logged-in user saves their profile so the daily job can recommend for them.

Note: the match % shown is now the TOPSIS **closeness coefficient** c — a number produced by a published formula, not a hand-tuned score.

6. Jury questions — ready answers

Q — What are you actually calculating? A Euclidean distance? Yes. For each plan we compute its **Euclidean distance to the ideal plan** (cheapest, most data) and to the anti-ideal, then a closeness score $c = s_{\square}/(s_{\square}+s_{\square})$. That is the TOPSIS method; we rank by c .

Q — Did you invent the formula? No. The ranking is **TOPSIS** (Hwang & Yoon, 1981) and the weights come from **Rank-Order Centroid** (Barron & Barrett, 1996) — both are established, published formulae, still surveyed today (Taherdoost & Madanchian, 2023). The *only* project-specific decisions are **which criteria the user may rank** and that the **budget is a hard ceiling**.

Q — Why not AI / collaborative filtering? They learn from a history of user behaviour we don't have at launch (the cold-start problem). TOPSIS is content-based — it ranks from the plan attributes and the user's stated profile alone, so it works from day one.

Q — Why TOPSIS and not a weighted sum, lexicographic ordering, or AHP? A plain weighted sum gives an unbounded score and is scale-sensitive; strict lexicographic ordering is too rigid (a tiny price gap erases any data advantage); AHP needs many pairwise comparisons users can't easily give. TOPSIS lets the criteria trade off smoothly, ranks against an explicit ideal, returns a bounded 0–1 score, and pairs naturally with ROC weights.

Q — How are the weights chosen — by hand? No. The user only **rank**s the criteria; ROC converts that ranking into weights (0.75 / 0.25 for two), which is the most accurate of the simple rank-based weighting formulae. So the weights are *derived*, not invented.

Q — Is it compensatory (can data offset price)? Yes — that's deliberate. A small price difference shouldn't erase a big data advantage. The user's priority is still respected through the ROC weights (price weighted 0.75 $\square$ data 0.25), so price dominates without being absolute.

Q — What if no plan fits? Budget is a hard ceiling, so the list is simply empty and the screen says so; the user can raise the budget or relax a filter.

Next guide: the search engine (token parser + FTS5). Tell me when you want it.